

Task 2: Dataset Selection

The dataset originates from Kaggle (<https://www.kaggle.com/datasets/kamilpytlak/personal-key-indicators-of-heart-disease>). According to the owner of the dataset, the dataset is collected by the Centers of Disease Control and Prevention (CDC) of the United States of America under the Behavioral Risk Factor Surveillance System (BRFSS) which collects the health status data of American residents through annual telephone surveys. The owner of the dataset cleaned the dataset and posted it on Kaggle. In this project, the 2022 dataset (without missing values) is used.

The dataset contains 246022 instances. The “State” attribute and the “RaceEthnicityCategory” attribute from the original dataset are removed because it is logically not relevant to the prediction task, which is predicting the risk of having a heart attack. The remaining number of attributes, including the label attribute “HadHeartAttack”, is 38. The type and brief description of each attribute of the dataset is listed in Table 1.

Table 1 Attributes in the dataset

Attribute	Type	Description
Sex	Nominal	Gender of the respondent (Male/Female).
GeneralHealth	Ordinal	Self-rated general health status (Excellent–Poor).
PhysicalHealthDays	Numerical	Number of days in past 30 when physical health was not good.
MentalHealthDays	Numerical	Number of days in past 30 when mental health was not good.
LastCheckupTime	Ordinal	Time since last medical checkup.
PhysicalActivities	Nominal	Whether respondent engaged in physical activity in past 30 days.
SleepHours	Numerical	Average hours of sleep per day.
RemovedTeeth	Ordinal	Number of permanent teeth removed due to decay or disease.
HadHeartAttack	Nominal	Whether respondent has ever had a heart attack (target variable).
HadAngina	Nominal	Whether respondent has been diagnosed with angina or coronary heart disease.
HadStroke	Nominal	Whether respondent has ever had a stroke.
HadAsthma	Nominal	Whether respondent has ever been diagnosed with asthma.
HadSkinCancer	Nominal	Whether respondent has ever been diagnosed with skin cancer.
HadCOPD	Nominal	Whether respondent has been diagnosed with chronic obstructive pulmonary disease.
HadDepressiveDisorder	Nominal	Whether respondent has ever been diagnosed with depression.
HadKidneyDisease	Nominal	Whether respondent has been diagnosed with kidney disease (excluding stones/infection).
HadArthritis	Nominal	Whether respondent has been diagnosed with arthritis.
HadDiabetes	Nominal	Whether respondent has been diagnosed with diabetes (or borderline).
DeafOrHardOfHearing	Nominal	Whether respondent is deaf or has serious hearing difficulty.
BlindOrVisionDifficulty	Nominal	Whether respondent has serious vision difficulty.
DifficultyConcentrating	Nominal	Whether respondent has serious difficulty concentrating or remembering.

DifficultyWalking	Nominal	Whether respondent has serious difficulty walking or climbing stairs.
DifficultyDressingBathing	Nominal	Whether respondent has difficulty dressing or bathing.
DifficultyErrands	Nominal	Whether respondent has difficulty doing errands alone.
SmokerStatus	Nominal	Smoking status of the respondent (never, former, current).
ECigaretteUsage	Nominal	E-cigarette usage status.
ChestScan	Nominal	Whether respondent has ever had a CT scan of the chest.
AgeCategory	Ordinal	Age group category (e.g., 18–24, 25–29, ...).
HeightInMeters	Numerical	Respondent's height in meters.
WeightInKilograms	Numerical	Respondent's weight in kilograms.
BMI	Numerical	Body Mass Index calculated from height and weight.
AlcoholDrinkers	Nominal	Whether respondent consumes alcohol.
HIVTesting	Nominal	Whether respondent has ever been tested for HIV.
FluVaxLast12	Nominal	Whether respondent received a flu vaccine in the past 12 months.
PneumoVaxEver	Nominal	Whether respondent ever received the pneumococcal vaccine.
TetanusLast10Tdap	Nominal	Whether respondent received a tetanus shot in the last 10 years (Tdap).
HighRiskLastYear	Nominal	Whether respondent was considered high-risk for any condition in the past year.
CovidPos	Nominal	Whether respondent tested positive for COVID-19.

Task 3: Problem Framing

The machine learning problem that we intend to solve is a supervised learning problem to predict whether a person is at high risk of having a heart attack based on his/her health information and daily habits. This is a supervised learning machine learning problem because there is a label in the dataset that indicates whether the patient has a heart attack. To be more specific, this supervised learning problem is a binary classification problem, whereby the target attribute is “HadHeartAttack” and the other attributes are predictor attributes. The labels of the instances are “Yes” or “No”, where “Yes” means that a person is at high risk of having a heart attack, whereas “No” means that the person is not at high risk of having a heart attack. In the context of its application, the trained model is able to help hospitals and health agencies to monitor the risk of their patient having a heart attack in the near future. The patient can then be called for early screening and diagnosis. The patient can be alerted that based on his current health information and daily habits, he/she is actually at high risk of having a heart attack.

Task 4: Data Preparation

The first data pre-processing done on the entire dataset is trying to remove any row that contains a missing value. However, there are no missing values in the dataset, so no rows are removed. The remaining number of rows is 246022. After that, label encoding is applied on all nominal attributes by using LabelEncoder from the Scikit Learn library. Ordinal encoding is applied to ordinal attributes by using OrdinalEncoder from the Scikit Learn library. Standard feature scaling is applied to numerical attributes by using StandardScaler from the Scikit Learn library. Refer to Table 1 to see which attributes are classified as nominal, ordinal or numerical.

Task 5: Experiment Setup

The experiments are divided into 4 sets, namely set 1, set 2 and set 3. Experiment set 1 is comparing machine learning algorithms, whereas experiment set 2 is selecting features, whereas experiment set 3 is ensemble learning. For all experiments the same dataset is used. The dataset is split to training set, development set and test set. Firstly, 20% of the dataset is used as the test set. Then, 20% of the

remaining dataset is used as the development set. Finally, the remaining dataset (about 60%) is used as the training set.

The class distribution of the label (HadHeartAttack) is unbalanced because there are 232587 instances of “No” and 13435 instances of “Yes”. So, accuracy is not suitable to be used as the performance metric because using accuracy in an imbalance dataset will cover up the low performance in predicting the minority class. The weighted average of F1 score is also not suitable because using weighted F1 will let the class with great majority dominate the F1 score. In this project, the class with a great majority is the “No” instances, with 94.54%. Instead, the macro average (simple average) of F1 score is a more suitable performance metric because it gives equal weight to the F1 score of each class. This gives a fairer evaluation on the performance of predicting both the “No” class and the “Yes” class accurately. To ensure consistency, all the experiment is using the same performance metric to evaluate the model, which is macro average F1.

Experiment Set 1

In this experiment set, four machine learning algorithms, namely decision tree, logistic regression, multi-layer perceptron and k nearest neighbour, are tried out. The hyperparameter is tuned by using GridSearchCV from the Scikit Learn library. The cv parameter is set as 3, which means the GridSearchCV is set to tune the hyperparameter using 3-fold cross validation. It is important to note that the performance metric chosen for evaluation in this project is macro average F1 score. So, the scoring parameter for GridSearchCV is set as “f1_macro”, so that all hyperparameters are tuned towards a higher macro average F1 score. Table 2 lists the hyperparameter tried during hyperparameter tuning and the best hyperparameter found.

Table 2 Experiment setup for hyperparameter tuning and best parameter found

Algorithm	Tried hyperparameters	Best hyperparameter
Decision tree	criterion: [gini, entropy, log_loss], splitter: [best, random], max_depth: [10, 30, 60], min_samples_split: [20, 80, 100], min_samples_leaf: [20, 80, 100], min_impurity_decrease: [0.01, 0.05], class_weight: [balanced, None]	criterion: gini, splitter: best, max_depth: 10, min_samples_split: 20, min_samples_leaf: 20, min_impurity_decrease: 0.01, class_weight: balanced
Logistic regression	penalty: [None, l2], class_weight: [balanced, None], solver: [lbfgs, newton-cg, newton-cholesky, sag]	penalty: None, class_weight: None, solver: lbfgs
Multi-layer perceptron	hidden_layer_sizes: [(25, 10, 5), (40, 20, 10, 5), (35, 15, 5)], activation: [logistic, tanh, relu]	hidden_layer_sizes: (25, 10, 5), activation: logistic
K nearest neighbour	n_neighbors: [5, 10, 20, 50, 100, 200, 300], weights: [uniform, distance], p: [1, 2]	n_neighbors: 5, weights: distance, p: 1

The model of each algorithm is trained using the best hyperparameter found and the train, validation and test macro average F1 are recorded. Table 3 shows the train, validation and test macro average F1 of each model.

Table 3 Validation and test macro average F1 of models

Algorithm	Train macro average F1	Validation macro average F1	Test macro average F1

Decision tree	0.719355	0.725234	0.730504
Logistic regression	0.654473	0.660109	0.657801
Multi-layer perceptron	0.657478	0.654855	0.658581
K nearest neighbour	0.999994	0.546034	0.549028

The performance of the models is determined by the test macro average F1. Based on Table 3, the decision tree model is the best performing model because it has the highest test macro average F1. This implies that the underlying function of the dataset matches the hypothesis space of the decision tree. K nearest neighbour is the worst performing model because it has the lowest test macro average F1. The reason for the poor performance is because the k nearest neighbour model overfits the training data. This is supported by the extremely high train macro average F1 and low validation macro average F1 and test macro average F1. The difference is at least 0.450966.

Experiment Set 2

In this experiment set, four feature selection approaches are used to select features. There are 6 numerical features and 31 categorical features in the dataset. Variance threshold and ANOVA filter are used to select 3 numerical features, whereas Chi squared and information gain are used to select 16 categorical features. Table 4 lists the features selected by using different feature selection methods.

Table 4 Features selected by different methods

Method	Features selected
Variance threshold	PhysicalHealthDays, HeightInMeter, WeightInKilograms
ANOVA filter	PhysicalHealthDays, WeightInKilograms, BMI
Chi squared	GeneralHealth, LastCheckupTime, RemovedTeeth, HadAngina, HadStrok, HadCOPD, HadKidneyDisease, HadArthritis, HadDiabetes, DeafOrHardOfHearing, DifficultyWalking, DifficultyDressingBathing, DifficultyErrands, ChestScan, AgeCategory, PnuemoVaxEver
Information gain	Sex, GeneralHealth, PhysicalActivities, RemovedTeeth, HadAngina, HadArthritis, HadDiabetes, DifficultyWalking, SmokerStatus, ChestScan, AgeCategory, AlcoholDrinkers, HIVTesting, FluVaxLast12, PnuemoVaxEver, TetanusLast10Tdap

Four sets of features are generated from the selected features by combining the numerical features and the categorical features. Each set of features are tested on different machine learning algorithms. The best hyperparameters of the machine learning models found in experiment set 1 are used. Table 5 shows the test macro average F1 for each model in each set of selected features.

Table 5 Test macro average F1 for each model in each set of selected features

Set	Combination of features	Test macro average F1 of decision tree	Test macro average F1 of logistic regression	Test macro average F1 of multi-layer perceptron	Test macro average F1 of k nearest neighbour
Set 1	Chi squared + Variance threshold	0.730504	0.649931	0.638695	0.587371
Set 2	Chi squared + ANOVA filter	0.730504	0.649082	0.635719	0.592635
Set 3	Information gain + Variance threshold	0.730504	0.652763	0.648445	0.578245

Set 4	Information gain + ANOVA filter	0.730504	0.653384	0.649628	0.581426
-------	---------------------------------	----------	----------	----------	----------

Based on Table 5, the decision tree achieved the same test macro average F1 for all sets of features, whereas logistic regression achieved the highest test macro average F1 for set 4, whereas multi-layer perceptron achieved the highest test macro average F1 for set 4, whereas k nearest neighbour achieved the highest test macro average F1 for set 2.

Comparing with the performance of the full set of features, the test macro average F1 of decision tree is the same, whereas the test macro average F1 of logistic regression decreases by 0.004417, whereas the test macro average F1 of multi-layer perceptron decreases by 0.008953, whereas the test macro average F1 of k nearest neighbour increases by 0.043607.

The reason for the performance of decision tree remains the same is the maximum depth of decision tree is set as 10, so the decision tree already performs feature selection by itself even when the full set of features is used. The reason for the performance of k nearest neighbour increases is k nearest neighbour typically does not perform well when the dimensionality is high. So, reducing the dimensionality can improve the performance of k nearest neighbour.

In overall, the model performance either remains the same, increases or decreases slightly. So, it can be said that the dimensionality can be reduced with minimal harm to the performance. Reducing dimensionality will reduce the computational resources needed to run the model, thus making the model more efficient. Importantly, this does not affect the performance significantly.

Experiment Set 3

In experiment set 3, three ensemble learning methods are tested, namely bagging, Ada Boost and hard voting. Set 4 in the features selected in experiment set 2. See Table 4 and Table 5 for the features in set 4. Set 4 is selected because it is the best set of selected features in experiment set 2. In bagging, decision tree, logistic regression, multi-layer perceptron and k nearest neighbour are used, whereas in Ada Boost, decision tree and logistic regression are used. In hard voting, the combination of logistic regression, multi-layer perceptron and k nearest neighbour and the combination of logistic regression and multi-layer perceptron are used. Table 6 lists the macro average F1 of all ensemble models.

Table 6 Test macro average F1 of ensemble models

Ensemble method	Algorithm(s)	Test macro average F1
Bagging	Decision tree	0.730504
	Logistic regression	0.654366
	Multi-layer perceptron	0.651464
	K nearest neighbour	0.577316
Ada Boost	Decision tree	0.730504
	Logistic regression	0.653384
Hard voting	Logistic regression, multi-layer perceptron, k nearest neighbour	0.633065
	Logistic regression, multi-layer perceptron	0.619448

Ensemble method, such as bagging and Ada Boost does not improve the performance of decision tree because the test macro average F1 is the same with experiment set 2 and experiment set 1. Nonetheless, decision tree is still the universal best performing model in this project.

Applying bagging on logistic regression improves the test macro average F1 by 0.000982 compared to experiment set 2, whereas applying Ada Boost on logistic regression does not change the test macro average F1 compared to experiment set 2. Applying bagging on multi-layer perceptron improves the test macro F1 by 0.001836 compared to experiment set 2. Applying bagging on k nearest neighbour decreases the macro average F1 by 0.004110.

Hard voting combining logistic regression, multi-layer perceptron and k nearest neighbour has higher performance than k nearest neighbour alone, but has lower performance than logistic regression alone and multi-layer perceptron alone. Hard voting combining logistic regression and multi-layer perceptron has lower performance than logistic regression and multi-layer perceptron alone.

This experiment shows that more complex model does not necessarily outperform simpler model. This is because in the cases where performance degraded, the degradation is significant, whereas in the cases where performance improved, the improvement is marginal.

Conclusion

In this project, several approaches to train a machine learning model for heart attack risk prediction are explored. In experiment set 1, decision tree, logistic regression, multi-layer perceptron and k nearest neighbour are tuned and compared. Decision tree is the best performing model. In experiment set 2, the dimensionality is reduced through feature selection approaches, namely variance threshold and ANOVA filter for numerical attributes and Chi squared and information gain for categorical attributes. The machine learning models are retrained using the selected features and it is found that reducing the number of features has minimal harm on the performance if the features are selected properly. In experiment set 3, several ensemble approaches, namely bagging, Ada Boost and hard voting, are applied on the machine learning algorithms. It is found that there is no significant improvement in the performance after applying ensemble. The universal best performing model is decision tree, which achieves test macro F1 of 0.730504 for all three experiment sets. The complexity of the decision tree models in each experiment set is different. The least complex one, which is the decision tree in experiment set 2, is chosen as the proposed model.